General Certificate of Secondary Education
2019

Digital Technology

Unit 4:

Digital Development Concepts

[CODE]

SPECIMEN

MARK SCHEME

		AVAILABLE MARKS
1	**(a)** C Examining variables within the program and checking their values for errors [1]	
	(b) D A value that can be assigned but remains the same throughout the program [1]	
	(c) D Converts the program code to binary format [1]	3
2	A program design solution can be developed using **algorithms** [1]. The designs are converted into sets of instructions for computers called **programs** [1]. Locations in memory called **variables** [1] are used to hold any data being processed.	3

3 (a) Level 0 [0]
Answer is not worthy of credit.

Level 1 ([1])
The candidate refers to one of the areas in context. The candidate makes limited use of spelling, punctuation and grammar. The meaning of the text is not always clear. The candidate displays a limited form and style appropriate to the question. The organisation of the answer is limited.

Level 2 ([2]–[3])
The candidate refers to both areas in context. The candidate makes satisfactory use of spelling, punctuation and grammar. The meaning of the text is usually clear. The candidate demonstrates a satisfactory form and style appropriate to the question. The organisation of the answer is satisfactory.

Level 3 ([4])
The candidate describes fully the two headings in context. The candidate makes good use of spelling, punctuation and grammar. The meaning of the text is always clear. The candidate demonstrates a good form and style appropriate to the question. The organisation of the answer is good.

Answers may include:

Abstraction
- Involves removing specific detail and focussing on general characteristics
- Involves creating a model/modelling the problem to be solved
- Involves the filtering out of characteristics that are not required
- One short relevant example of abstraction

AVAILABLE MARKS

Decomposition

- Involves breaking larger/complex problems into smaller problems
- Smaller problems are easier to understand/large problems are difficult to understand
- Solutions to smaller problems can be designed independently
- Solutions to smaller problems can be brought together to form one solution
- One short relevant example of decomposition [4]

(b) (i)

Variable	Data Type	
percentageCommission	Integer	[1]
valueOfSales	Real/Double/Float	[1]

[2]

(ii) Correct structure chosen, IF-Statement[1]

```
If valuesOfSales >=1 and valueOfSales<=999 [1]
    percentageCommission=0.01 [1]
    commission= percentageCommission * valueOfSales [1]
elseIf valuesOfSales >=1000 and valueOfSales<=1999 [1]
    percentageCommission=0.05
    commission = percentageCommission * valueOfSales
elseIf valuesOfSales >=2000 [1]
    percentageCommission=0.1
    commission = percentageCommission * valueOfSales
```

Allocate mark once only for assigning a value to percentageCommission
Allocate mark once only for calculating commission [6]

(iii)

Black Box Testing
Any **two** from:

- Ensuring that valid inputs are accepted/ invalid inputs rejected
- Ensuring that a correct output is returned
- The internal methods of the system are never known to the tester
- Ensures that modules work well at system level [2]

White Box Testing
Any **two** from:

- The internal components of a system are tested
- The program code and logic is trialled using test data
- Programming skills are required to test the objects that make up an application [2]

Unit Testing
Any **two** from:

- Units sections or modules of code [1] are tested independently [1]
- Each module will have its own test plan [1] [2]

(iv)

valueOfSales Any **three** acceptable data values [3] Eg	Reason for Test Any **three** acceptable validation checks [3] Eg	Expected Outcome Any **three** suitable and corresponding error messages [3] Eg
Enter key	Null data (Presence check)	Error message
1	Extreme value	Data accepted
-200	Outside of range (Range Check)	Error message
12w	Type Check	Error message
999	Extreme Value	Data accepted
224	Valid data	Data accepted

[9]

AVAILABLE MARKS

27

AVAILABLE MARKS

4 (a) **Level 0 [0]**
Answer is not worthy of credit.

Level 1 ([1]–[2])
The candidate names one, two or three validation checks in context. The candidate makes limited use of spelling, punctuation and grammar. The meaning of the text is not always clear. The candidate displays a limited form and style appropriate to the question. The organisation of the answer is limited.

Level 2 ([3]–[4])
The candidate describes two or three validation checks in context. The candidate makes satisfactory use of spelling, punctuation and grammar. The meaning of the text is usually clear. The candidate demonstrates a satisfactory form and style appropriate to the question. The organisation of the answer is satisfactory.

Level 3 ([5]–[6])
The candidate describes fully three validation checks in context. The candidate makes good use of spelling, punctuation and grammar. The meaning of the text is always clear. The candidate demonstrates a good form and style appropriate to the question. The organisation of the answer is good.

Answers may include a suitable validation check with associated explanation and example such as:

- **Range** – to ensure data entered is within an acceptable range for example percentages should be entered in the range 1-100.
- **Type** – to ensure the data entered is of the correct data type for example percentages should be numeric whereas names would be of type string.
- **Format** – to ensure that data follows a particular pattern/layout for example, a postcode must have 2 letters, 2 numbers, a space(optional), 1 number, 2 letters.
- **Presence** – to check that data has been entered. For example, when completing an online form a presence check may be entered to ensure that data has been entered into the email field.
- **Length** – to ensure that data entered does not exceed a particular number of characters in length/is at least a certain number of characters in length. For example, a length check could be used to ensure that a password is at least 8 characters long. [6]

AVAILABLE MARKS

(b)

Sample C#

```
do
    {
    okFlag = true;                                    [1] flag assignment

    Console.Write("Enter Dish Type in the range 1-5 \t");
    dishType = Convert.ToInt32(Console.ReadLine());

    if ((dishType < 1) || (dishType > 5))             [1] for if condition
      {
        Console.WriteLine("Error - out of range");
        okFlag = false;                    [1] for flag re-assignment
      }
    }
while (okFlag == false);                              [1] loop condition
```

[4]

(c) (i)

Any **two** from (note any valid string function is acceptable with a valid explanation for 2 marks):

CompareTo()	Compare two strings and returns integer value as output. It returns 0 for true and 1 for false.
Contains()	Checks whether specified character or string is exists or not in the string value.
EndsWith()	Checks whether specified character is the last character of string or not.
Equals()	Compares two string and returns Boolean value as output.
IndexOf()	Returns the index position of first occurrence of specified character.
ToLower()	Converts String into lower case based on rules of the current culture.
ToUpper()	Converts String into Upper case based on rules of the current culture.
Insert()	Insert the string or character in the string at the specified position.
Length	Returns length of string.
Remove()	Deletes all the characters from beginning to specified index position.
Replace()	This method replaces the character.
Split()	Splits the string based on specified value.
StartsWith()	Checks whether the first character of string is same as specified character.
Substring()	Returns substring.
Trim()	Removes extra whitespaces from beginning and ending of string.

[4]

(ii) Any **two** from:

Use the Length function [1] /Use a condition [1]
(name.Length>=6 [1] and name.Length<=30 [1])

[2]

AVAILABLE MARKS

16

5 (a)

	Tick (✓)
A) 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20	
B) 0 3 6 9 12 15 18 21	
C) 3 6 9 12 15 18	
D) 0 3 6 9 12 15 18	✓
E) 3 6 9 12 15 18 21	

[1]

(b) (i)

B- This is the second statement [1]

(ii)

C- This is the third statement [1]

3

6 (a)

```
int[] pounds = new int [10]
```

correct array size [1]
correct data type [1]
correct name for array [1]

[3]

(b)

```
for (k = 0; k<10; k++)
        pounds[k] = 0;
```

Use of for loop [1]
Correct upper and lower bounds [1]
Correct assignment statement [1]

[3]

(c) Algorithm acceptable
(Marker instruction: upperbound = 10, lowerbound = 1 OR upperbound = 9, lowerbound = 0)

```
SET COUNTER = 0
DECLARE ARRAY POUNDS
FOR COUNTER = lowerbound TO upperbound [1]
        GET POUNDSVALUE [1]
        POUNDS [COUNTER] = POUNDSVALUE [1]
END FOR
```

or

AVAILABLE MARKS

```
SET COUNTER = 0
DECLARE ARRAY POUNDS
WHILE COUNTER < upperbound [1]
                    GET POUNDSVALUE [1]
                    POUNDS [COUNTER] = POUNDSVALUE [1]
                    COUNTER = COUNTER +1
END WHILE
OR  DO.....WHILE also acceptable [1]
```

or

```
c# code

for (k = 0; k<10; k++) [1]
{
     pounds[k] [1] = Convert.ToInt32(Console.ReadLine()); [1]
}
```

[3] 9

AVAILABLE MARKS

7 (a)

Operations	
Performing a series of operations one after the other.	**Sequence** Selection Iteration [1]
The calculation of an employee's tax payment depends on their salary level.	Sequence **Selection** Iteration [1]
The program will continue to add the cost of a basket of items until the basket is empty.	Sequence Selection **Iteration** [1]

[3]

(b) (i) Global variable: a variable which can be seen/used/accessed anywhere within an application/program [1] [1]

(ii) Local variable: a variable whose scope is limited [1] to a particular module/method within an application [1] [2]

(c) (i) Any **two** from:

- A class serves as a template/blueprint for creating an object
- Each object is created[instantiated] from a single class
- Each class can instantiate many objects [2]

(ii) An object is an instance of a class [1] with variables, functions and data structures [1] [2]

(d) Any **four** from:

- Auto completion of statements [1] suitable explanation [1]
- Syntax checks with colour coding[1] suitable explanation [1]
- Bracket matching [1] suitable explanation [1] [4] 14

8 (a) A list of characters recognised by the computer hardware and software and represented by a number. [1]

(b) ASCII uses the numbers 0 through 127 [1] to represent all English characters [and special control characters] [1]
Each letter is assigned a number [1]/for example 'A' is 65. [3]

(c) Unicode consists of more than 109000 characters ASCII consists of 128 [1]
Unicode uses 8, 16 or 32 bit characters based on different presentations while ASCII is a seven-bit encoding formula [1] [2] 6

AVAILABLE MARKS

9 **(a)** **(i)** 36/2 = 18 r 0
18/2= 9 r 0
9/2= 4 r 1
4/2 = 2 r 0
2/2 = 1 r 0
1/2 = 0 r 1

Calculation [1]
Answer in six bits 100100 / using 8 bits 00100100
Accurate answer [1]

or

use column base 2 solution[1] and check answer[1]:

128	64	32	16	8	4	2	1
0	0	1	0	0	1	0	0

(1x32) + (1x4) = 36 [2]

(ii) 22/16 = 1 r 6
1/16 = 0 r 1
Answer 16 hex

Calculation [1]
Accurate answer [1]
Mention base hexadecimal [1]

or

Use column base 16 solution [2] and check answer [1]:
16 1
1 6

(1x16) + (6x1)=22 [3]

(iii) Used to simplify how binary is represented [1]
For example the binary number **11010100** would be **D4 in hex**/ suitable example [1].

[2]

(b) **(i)**

	0	1	1	0	0	0	1	0
+	1	1	0	1	0	1	0	0
(1)	0	0	1	1	0	1	1	0

Accurate calculation [1]
Overflow [1]

[2]

(ii) Any **two** from:

An 8-bit overflow error will occur [1]
The overflow digit will be dropped as it cannot be stored [1]
The result of the calculation will be stored incorrectly [1] [2]

(c)

OR Gate			
A	**B**	**X**	
0	0	0	
0	1	**1**	[1]
1	0	**1**	[1]
1	1	1	
AND Gate			
A	**B**	**X**	
0	0	0	
0	1	**0**	[1]
1	0	**0**	[1]
1	1	1	

[4]

AVAILABLE MARKS

15

10 (a) (i)

Name	Bubble Sort [1]	Insertion Sort [1]
Loop Max 2 marks	Uses **two** nested for loops [1] **or** The outer loop runs n times (number of elements in array), and the inner loop runs n/2 times [2] **or** Reference to how the loop is used to process the elements: Bubble sort repetitively compares adjacent pairs of elements and swaps if necessary [2]	Uses **two** nested for loops. [1] **or** The outer loop runs n times (number of elements in array), and the inner loop runs n/2 times [2] **or** Reference to how the loop is used to process the elements: Insertion sort maintains a sorted sub-array, and repetitively inserts new elements into it. Using two loops [2]
Efficiency Max 2 marks	**Two** from: Bubble sort will carry out a set number of comparisons [1] and is useful for small sets of data [1] Passes through the array a number of times [1]	**Two** from: Insertion sorts only pass through the array only once [1] It is adaptive [1] efficient for data sets that are already partly/ substantially sorted [1]

[5]

(ii) If (studentHeight[i] > studentHeight[i+1])
or
If (StudentHeight[i-1] > temp)

Correct condition [1]
Correct array element index [1] [2]

(iii) studentHeight (Bubble Sort)

AVAILABLE MARKS

155	152	156	151	153

Pass 1

152	155	151	153	156

[1]

Pass 2

152	151	153	155	156

[1]

Pass 3

151	152	153	155	156

[1]

Pass 4 : as pass 3 [1]
Pass 5 : as pass 3 [1] [5]

or

studentHeight (Insertion Sort)

155	152	156	151	153

Pass 1

152	155	156	151	153

[1]

Pass 2

152	155	156	151	153

[1]

Pass 3

152	155	156	151	153

[1]

Pass 4

151	152	155	156	153

[1]

Pass 5

151	152	153	155	156

[1] [5]

				AVAILABLE MARKS
	(b)	Any **three** from: • Binary Search [1] is applied to a sorted list [1] • The search value is compared with elements in the middle position of the array [1] • If the value is less than the middle element, then it must lie in the lower half of the array and if it's greater than the element then it must lie in the upper half of the array [1] • Binary Search is useful when there are large numbers of elements in an array as the number of comparisons is reduced [1]	[3]	15
11	**(a)**	**(i)** A named module/section of code written by the user [1] It may have parameters [1] It will have a return value [1]	[3]	

		(ii) [private] double findAverage (int num1, int num2, int num3)
		{
		
		double myaverage=0.0;
		myaverage=(num1+num2+num3)/3;
		return myAverage;
		}
		
		Any **three** from:
		
		• Function heading parameters
		• Function heading return type
		• Return statement
		• Calculation of average
		• Declaration of double/real/float variable inside function
			[3]	

		(iii) [private] int findProduct (int num1, int num2, int num3)
		{
		
		int myProduct=0;
		myProduct=(num1*num2*num3);
		return myProduct;
		}
		
		Any **three** from:
		
		• Function heading parameters
		• Function heading return type
		• Return statement
		• Calculation of product
		• Declaration of double/real/float variable inside function
			[3]	9

			Total	**120**